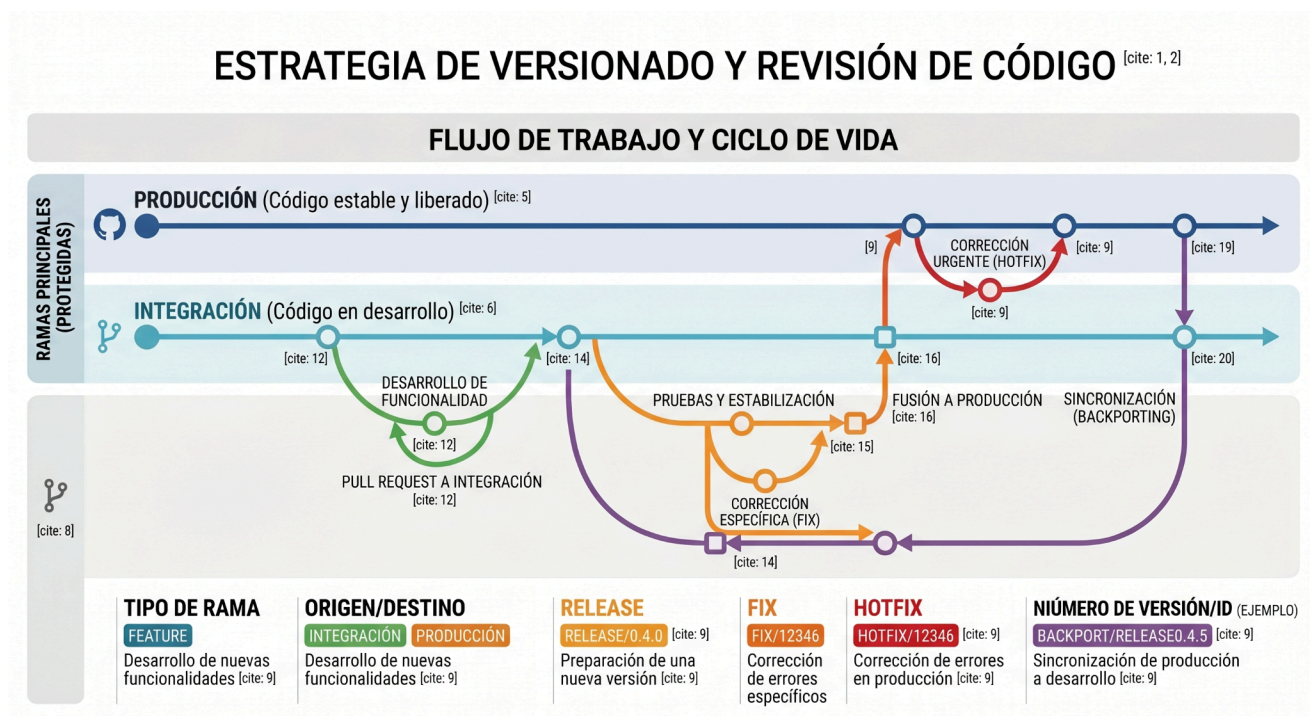




4. Política de ramas y proceso de revisión de pull requests

La política de ramas y el proceso de revisión de pull requests garantizan orden, trazabilidad y calidad en el desarrollo colaborativo.

Esta sección define el modelo de ramificación y el flujo de trabajo para el control de versiones, garantizando un ciclo de vida de desarrollo estructurado y estable.



Ramas Principales (Protegidas)

- **MASTER:** Rama de producción. Contiene código estable y liberado. Está protegida y solo recibe fusiones desde ramas RELEASE/.
- **DEVELOP:** Rama de integración. Contiene el código en desarrollo para la próxima versión. Está protegida y recibe integraciones de FEATURE/ y BACKPORT/.

Ramas de Soporte y Nomenclatura

Tipo de Rama	Origen	Destino	Descripción
feature/<nombre>	develop	develop	Desarrollo de nuevas funcionalidades (ej. feature/userCreation).



release/<versión>	develop	master	Preparación de una nueva versión a producción (ej. release/0.4.0).
fix/<id>	release/ o develop	release/ o develop	Corrección de errores específicos (ej. fix/12346 o fix/sdfsdf).
hotfix/<id>	master	master	Corrección de errores en producción (ej. hotfix/12346 o hotfix/nullpointerfix).
backport/<versión>	master	develop	Sincronización de cambios de producción hacia desarrollo (ej. backport/0.4.5).

Flujo de Trabajo y Ciclo de Vida

Desarrollo de Funcionalidades

Todo nuevo desarrollo inicia desde **develop** creando una rama **feature/**. Al finalizar, se realiza un Pull Request hacia **develop**.

Preparación de Release y Correcciones

1. Se extrae una rama **release/** desde **develop** cuando las funcionalidades están listas para la próxima versión.
2. Si se detectan anomalías durante las pruebas, se crean ramas **fix/** que se fusionan sobre la rama **release/** correspondiente (ej. **fix/12346** → **release/0.4.0**).
3. Una vez estabilizada y aprobada, la rama **release/** se fusiona directamente hacia **master** (ej. **release/0.4.5** → **master**).

Estrategia de Sincronización (Backporting)

Para asegurar que develop incorpore de manera estricta todas las correcciones aplicadas durante la fase de release en producción, se emplea el siguiente mecanismo de backporting:

- Se deriva una rama backport directamente desde master (ej. master → backport/0.4.5).



- Esta rama se fusiona hacia develop (backport/0.4.5 → develop), asegurando la paridad del código base.

Proceso de revisión de pull requests

Es una herramienta de equipo para:

- Compartir conocimiento y onboarding continuo
- Detectar bugs, edge cases y deuda técnica temprano
- Mantener consistencia en arquitectura, seguridad y estilo
- Elevar el nivel técnico de todo el equipo
- Crear responsabilidad compartida sobre el código

Regla de oro: Se revisa el código, no a la persona.

Sección	Qué incluir	Ejemplo
Título	[Tipo] Ticket: Descripción corta	feature/USER-123: añadir validación de email
Descripción	Qué hace, por qué se hace, impacto	Implementa regex + test unitarios. para Evitar registros inválidos en auth.
Contexto o Background	Enlace a ticket, decisiones técnicas, alternativas descartadas	Ver JIRA-456. Se eligió Clode Clean por acuerdo del team
Cómo probarlo	Pasos claros, credenciales o links, URLs, mocks	1. Levantar npm run dev2. Ir a/register3. Usar emailtest@ → ver error
Screenshots/GIFs	Obligatorio si hay UI, logs o flujos visuales	![registro_valido.gif](link)
Checklist de Auto-Verificación	Marcar solo si aplica	- [x] Tests pasan localmente - [x] CI verde - [x] Sin console.log de debug
Tickets/Issues	Links a Jira	Closes #89, Relates to JIRA-456

Flujo del Proceso

1. Autor crea la rama → trabaja → hace commits atómicos → sube a GitHub
2. Autor abre PR, llena plantilla, asigna 1-2 revisores, verifica CI